Sicurezza - Domande d'Esame
Sapienza Università di Roma
A.A 2024-2025

Indice

1 Prova d'esame - 16 giugno 2025 **3**
- 1.1 Prof. Visconti . . . 3
- 1.2 Prof. D'Elia . . . 4

2 Prova d'esame - 18 luglio 2025 **7**
- 2.1 Prof. Visconti . . . 7
- 2.2 Prof. D'Elia . . . 8

3 Altre domande - Visconti **10**

4 Altre domande - D'Elia **21**

Raccolta di domande d'esame per il corso di Sicurezza dei professori D'Elia e Visconti. Corso erogato dal 2025, pochi appelli disponibili. Le risposte sono volutamente corte per indicare gli argomenti pricipali.

1 Prova d'esame - 16 giugno 2025

1.1 Prof. Visconti

Quesito 1. **Sia B un block cipher che cifra un blocco di 256 bit di plaintext usando una chiave K di 128 bit casuali e dando in output un blocco di 256 bit di ciphertext.**
Dato un Array A di n bit, denotiamo con $A[X-Y]$ i bit dall'indice X all'indice Y.
Denotiamo con "||" l'operatore di concatenazione di stringhe.
Si consideri la cifratura di un messaggio M di 1024 bit come segue:
$Enc(K, M) =$
$B(K, M[0-255]) \;||\; B(K, M[256-511]) \;||$
$B(K, M[512-767]) \;||\; B(K, M[768-1023])$
Commentare la bontà della precedente cifratura.

Risposta:

- **Cifratura**: ECB (Electronic Code Book) mode (Visconti, sec. 3.1 *Electronic Code Book (ECB)*, pag. 14).
- **Bontà**: Totalmente insicuro (**non CPA-secure**). La funzione è **deterministica**: blocchi di plaintext uguali generano blocchi uguali (es. Pinguino Tux visibile).

Quesito 2. **Spiegare brevemente perché con chiave K casuale di 256 bit, $SHA256(K, M)$ in generale non è una buona funzione di MAC, indicando come può essere violata l'unforgeability.**

Risposta:

- **Validità**: Non è un buon MAC per via del **Length extension attack** (Visconti, sec. 6.3.4 *Uso di CRHF per MAC*, pag. 23).
- **Unforgeability**: L'attaccante può estendere l'hash aggiornandone lo stato interno per calcolare il MAC fittizio di un nuovo messaggio $M||M'$, forgiando la firma.

Quesito 3. **Spiegare in poche righe l'attacco Human-in-the-middle all'accordo di chiavi chiamato Diffie-Hellman e indicare perché non è realizzabile in TLS.**

Risposta:

- **MITM**: L'attaccante si interpone concordando due chiavi DH distinte con Alice e Bob decifrando e recifrando il traffico senza farsi notare (Visconti, sec. 9.1.1 *Diffie-Hellman e Man-in-the-Middle Attack*, pag. 41).
- **In TLS**: Si evita chiedendo che l'accordo DH sia autenticato dai **certificati X.509** e relative firme digitali (PKI).

Quesito 4. **Quando ci si identifica mediante login e password, queste vengono inviate in un canale sicuro al server; in uno scenario in cui si vuole limitare la possibilità di un dictionary attack, quali azioni compie quindi il server per controllare che la password sia corretta? (nota: la risposta dovrebbe consistere di poche righe che dettagliano questo task specifico del server, e non serve coinvolgere PBKDF2)**

Risposta:

- **Memorizzazione**: Il server non salva la password in chiaro, ma una coppia `salt, H(salt ‖ password)`, dove `salt` è una stringa casuale unica per ogni utente (Visconti, sec. 6.3.6 *Autenticazione password*, pag. 24).
- **Verifica al login**: Al momento dell'autenticazione il server:
  (a) Recupera il `salt` associato all'utente dal database.
  (b) Calcola `H(salt ‖ password_inserita)`.
  (c) Confronta il risultato con l'hash memorizzato: se coincidono, la password è corretta.
- **Mitigazione dictionary attack**: L'uso del salt impedisce l'efficacia di rainbow table precomputate, poiché ogni password è hashata con un valore casuale diverso, costringendo l'attaccante a ricalcolare il dizionario separatamente per ogni utente.

Quesito 5. Spiegare cosa fanno i seguenti comandi:

```
openssl dhparam -out dhparam.pem 2048
openssl genpkey -paramfile dhparam.pem -out private_key.pem
openssl pkey -in private_key.pem -pubout -out DHtosend.pem
openssl pkeyutl -derive -inkey private_key.pem -peerkey DHreceived.pem -
    out secret.bin
```

Risposta:

- **dhparam**: Genera i parametri (e.g. numero primo grande) per Diffie-Hellman a 2048 bit.
- **genpkey**: Genera la chiave privata DH locale usando quei parametri.
- **pkey**: Estrae dalla chiave privata la porzione **pubblica** DH per inviarla al Peer.
- **pkeyutl**: Effettua la derivazione del **shared secret** fondendo la nostra chiave privata e la chiave pubblica ricevuta (`DHreceived.pem`).

1.2 Prof. D'Elia

Quesito 6. *(se possibile, rispondere entro 8 righe)*
Definire cosa è un threat model per un sistema software. Discutere le strategie che si possono intraprendere in risposta a ciascun threat in fase di design e le loro implicazioni.

Risposta:

- **Definizione**: Quali asset si proteggono, vettori di minaccia e capacità dell'avversario (D'Elia, *Modello di Minaccia*, sec. 2.3).
- **Design Strategie**: Scegliere mitigazioni che forzano un comportamento o accettare i rischi. Implicazioni: costi sistemistici e calo di performance/usabilità (crittografia, isolamento) per garantire i vincoli CIA.

Quesito 7. *(se possibile, rispondere entro 8 righe)*
Definire gli approcci di blackbox testing e whitebox testing per un software sottoposto a dynamic testing. Menzionare cosa è necessario conoscere riguardo al dominio di input del software ed alla sua implementazione per poterli utilizzare, poi confrontarli brevemente.

Risposta:

- **Black-box** (D'Elia, sec. 3.2.4 *Distinzione tra test dinamici: Black-box e White-box*, pag. 21; sec. 3.2.5, pag. 22): Testa il comportamento dalle specifiche (non vede il codice). **Pro**: Vede il sistema come l'utente, isolato dai bias dell'implementazione. **Contro**: Bassa copertura del codice interno, non garantisce alta copertura del codice, potrebbe non triggerare bug in path profondi.
- **White-box** (D'Elia, sec. 3.2.6 *Tecniche di test dinamico: esempio di white-box*, pag. 22): Analizza ed esplora l'implementazione per massimizzare la copertura. **Pro**: Scoperta di dead-code e si triggerano *code paths* complessi. **Contro**: Costoso, soffre di *implementation bias*.

Quesito 8. *(se possibile, rispondere entro 10 righe)*
Descrivere funzionamento e obiettivo dell'Address Space Layout Randomization. Discutere cosa si rende necessario per applicare ASLR sia a livello di librerie offerte dal sistema operativo che per le singole sezioni di memoria in uso da uno specifico programma.

Risposta:

- **ASLR**: Randomizza la disposizione (indirizzi) delle strutture (Stack, Heap, Code, Librerie) per rendere impossibili attacchi come ROP o Return-To-Libc basati su address previsti (D'Elia, sec. 6.6.5 *Mitigazione: ASLR*, pag. 61).
- **Applicazione**: Supporto Kernel/OS ASLR per randomizzare le zone di mapping per processi compatibili (Position Independent Executable - PIE, D'Elia, sec. 6.7 *PIE/PIC*, pag. 62).

Quesito 9. *(se possibile, rispondere entro 10 righe)*
Descrivere il ruolo del coverage feedback in un coverage-guided fuzzer. Discutere quale può essere l'impatto su aspetti chiave del fuzzer (es., sensitivity, queue size) di utilizzare un block coverage feedback (ossia una funzione di feedback che distingue l'identità di ogni basic block attraversato dall'esecuzione) al posto dell'edge coverage feedback.

Risposta:

- **Ruolo del coverage feedback**: Identificare gli input (seed mutations) che spingono l'esecuzione in nuovi path del programma da salvare in coda (D'Elia, sec. 7.3.4 *Coverage Feedback*, pag. 74; sec. 7.3.5 *Coverage Map*, pag. 74).
- **Block vs Edge coverage**:
  - Il *block coverage* traccia solo se un basic block è stato raggiunto, indipendentemente da quale predecessore lo ha attivato. Ha **sensitivity più bassa**: due input che arrivano allo stesso blocco da rami diversi sono considerati identici, quindi uno solo finisce in coda. La **queue size** risulta più piccola ma meno informativa, rischiando di perdere percorsi distinti.
  - L'*edge coverage* cattura invece la **transizione** specifica tra due blocchi ($A \to C$ vs $B \to C$), quindi ha **sensitivity più alta**: input che raggiungono lo stesso blocco da strade diverse vengono entrambi considerati interessanti. La **queue size** cresce di più rispetto al block coverage, ma in modo controllato grazie ai bucket di hit count, garantendo una migliore esplorazione del programma senza far esplodere la coda.
  - L'edge coverage è preferita nei fuzzer moderni (es. AFL++) proprio perché bilancia sensibilità e overhead meglio del block coverage, evitando sia la perdita di percorsi rilevanti sia la saturazione della coda.

Quesito 10. *(se possibile, rispondere entro 10 righe)*
Descrivere che tipo di memory-related bug è presente nel seguente codice. Spiegare quale tipo di sanitizer potrebbe individuarlo e, se sì, indicare durante l'esecuzione di quale linea di codice ciò può avvenire e perché.

```c
struct User {
    char name[32];
    char role[16];
    int is_admin;
    char padding[64];
};

void send_data(const char *buf, size_t len) {
    fwrite(buf, 1, len, stdout); // Simulated send (e.g., to a socket)
}

void foo() {
    struct User u;
    strcpy(u.name, "alice");
    strcpy(u.role, "user");
    u.is_admin = 0;
    send_data((char *)&u, sizeof(u));
}
```

Risposta:

- **Bug**: *Information disclosure / Uninitialized memory leak.* Il padding strutturale e i bytes rimanenti degli array `name`/`role` non sono mai azzerati prima del flush in stream.
- **Sanitizer**: **MemorySanitizer (MSan) (D'Elia, sec. 5.4.1, pag. 45)** la intercetterebbe. Si *triggera a runtime* sulla riga `send_data((char*)&u, ...)`, dato che lì l'uso di memoria non inizializzata compromette la confidenzialità (*Uninitialized Read*).

2 Prova d'esame - 18 luglio 2025

2.1 Prof. Visconti

Quesito 1. Sia B un block cipher che cifra un blocco di 256 bit di plaintext usando una chiave K di 128 bit casuali e dando in output un blocco di 256 bit di ciphertext. Sia K una stringa di 128 bit casuali.
Dato un Array A di n bit, denotiamo con $A[X-Y]$ i bit dall'indice X all'indice Y.
Denotiamo con "||" l'operatore di concatenazione di stringhe.
Si consideri la cifratura di un messaggio M di 1024 bit come segue:
$Enc(K, M, r)$**:**
Prendi una random string r di 256 bit
$c_1 = B(K, r)$
$c_2 = B(K, c_1 \oplus M[0-255])$
$c_3 = B(K, c_2 \oplus M[256-511])$
$c_4 = B(K, c_3 \oplus M[512-767])$
$c_5 = B(K, c_4 \oplus M[768-1023])$
Return $r \mid\mid c_2 \mid\mid c_3 \mid\mid c_4 \mid\mid c_5$
Commentare la bontà della precedente cifratura. NOTA: $\oplus = XOR$

Risposta:

- **Cifratura**: CBC (Cipher Block Chaining) mode (Visconti, sec. 3.2 *CBC mode*, pag. 15). r è l'IV casuale, trasmesso in chiaro, e il suo valore cifrato $c_1 = B(K, r)$ inizializza il chaining.
- **Sicurezza CPA**: Sicuro (CPA-secure) in quanto la cifratura dei blocchi è randomizzata dall'IV.
- **Difetti (Integrità)**: Manca un MAC (AEAD, Visconti, sec. 5.1 *AEAD*, pag. 21, o Encrypt-then-MAC), quindi è vulnerabile a *Chosen Ciphertext Attack (CCA)* o modifiche di bit da attaccanti attivi.

Quesito 2. Spiegare brevemente se $SHA256(K, M)$ è utilizzabile correttamente (e sotto quale assunzione) come funzione di MAC quando i messaggi di cui deve essere calcolato il MAC hanno tutti la stessa lunghezza.

Risposta:

- **Validità**: Funziona efficacemente solo per messaggi di **lunghezza fissa** rigida tra host.
- **Problema**: Se venisse usato in uno scenario che necessita di validare lunghezze arbitrarie, un attaccante potrebbe forgiare firme con il **Length Extension Attack** estendendo lo stato interno dell'hash, che nelle CRHF come SHA-256 non possiede meccanismi interni che lo prevengono (Visconti, sec. 4.1.3 *Attacco per schema a lunghezza fissa*, pag. 19).

Quesito 3. Mostrare come funziona l'accordo di chiavi chiamato Diffie-Hellman.

Risposta:

- **Scopo**: Meccanismo di key exchange su un canale di comunicazione insicuro (Visconti, sec. 8.6 *Diffie-Hellman*, pag. 38).
- **Funzionamento**: Entrambi scelgono un valore segreto (x, y) e si scambiano i valori calcolati su un gruppo finito prestabilito (g^x, g^y). La chiave unificata diviene g^{xy}.

- **Assunzione**: Sicurezza basata sulla difficoltà computazionale del problema del *Logaritmo Discreto* in domini enormi e, più propriamente, al problema DH Decisional (DDH/CDH).

Quesito 4. Spiegare brevemente che problema vuole mitigare PBKDF2 e come agisce.

Risposta:

- **Problema mitigato**: Difende l'hash delle password da **brute-force** o **dictionary attack** precomputati e tabelle rainbow.
- **Come agisce**: Utilizza un *salt* randomico persistito e poi applica ciclicamente una *Pseudorandom Function (PRF)* al risultato per **C iterazioni**. Dilata quindi i tempi computazionali di verifica (Visconti, sec. 6.3.7 *Cifrare i file: PBKDF2*, pag. 25).

Quesito 5. Spiegare cosa fanno i seguenti comandi:

```
1 openssl enc -e -aes-256-ctr -in test.txt -K 1234567890
      abcdef1234567890abcdef1234567890abcdef1234567890abcdef -iv
      1234567890abcdef1234567890abcdef -out c.bin
2 openssl enc -d -aes-256-ctr -K 1234567890
      abcdef1234567890abcdef1234567890abcdef1234567890abcdef -iv
      1234567890abcdef1234567890abcdef -in c.bin
```

Risposta:

- **Comandi**: Cifrano (`-e`) e decifrano (`-d`) il file mediante OpenSSL rispettivamente su `test.txt` e `c.bin`.
- **Algoritmo**: AES con lunghezza chiave a 256 bit elaborato in modalità **CTR (Counter mode)** facendolo lavorare parallelamente come uno *stream cipher* (Visconti, sec. 3.3 *Counter (CTR) mode*, pag. 16 & sec. 7.2.1 *AES-256-CTR*, pag. 28).
- **Parametri**: Vengono dichiarate la medesima Key a 256-bit (`-K`) e l'Initialization Vector a 128-bit (`-iv`) in base 16.

2.2 Prof. D'Elia

Quesito 6. *(se possibile, rispondere entro 8 righe)*
Nel contesto del threat modeling, descrivere le principali differenze tra un attaccante remoto (network user) e un attaccante locale (colocated user). Discutere come queste differenze influenzano l'attack surface e le potenziali contromisure da adottare in risposta.

Risposta:

- **Attaccante Remoto**: Tenta violazioni dalla rete (D'Elia, *Utente di rete*, sec. 2.3.1). *Attack Surface*: Manipolazione pacchetti e richieste Web. *Contromisure*: SSL/TLS, packet filtering/firewalling, rate limit.
- **Attaccante Locale (Co-locato)**: Esegue codice sulla stessa macchina della vittima. (D'Elia, *Utente co-locato*). *Attack Surface*: Lettura side-channels come memorie, cache, CPU ed IPC. *Contromisure*: Isolamento rigoroso, compartimentalizzazione dei flussi, Least Privilege.

Quesito 7. *(se possibile, rispondere entro 8 righe)*
Definire gli approcci di static testing e dynamic testing per un sistema software. Discutere per ciascun approccio due punti di forza e due punti di debolezza.

Risposta:

- **Static Testing** (D'Elia, sec. 3.2.2 *Tecniche di test statici*, pag. 21): Nessuna esecuzione, analisi astratta del codice-sorgente o semantica ASM. **Pro**: Può analizzare tutte le parti del codice senza eseguirlo. Trova logiche occulte. **Contro**: Costretti ad operare sovra-approssimazioni comportando eccessivi **falsi positivi**.
- **Dynamic Testing** (D'Elia, sec. 3.2.3 *Tecniche di test dinamico*, pag. 21): Rilevazione dei fault monitorando un'effettiva esecuzione. **Pro**: I fault rilevati sono effettivi (**bassissimo tasso di falsi positivi, poiché i bug rilevati sono concretamente riproducibili.**). Analizza il programma secondo stati realistici in runtime. **Contro**: **Copertura** limitata (non valuta i rami dove l'input non li conduce scovando **falsi negativi**).

Quesito 8. *(se possibile, rispondere entro 10 righe)*
Descrivere funzionamento e obiettivo di Data Execution Prevention. Discutere se e come DEP può ostacolare un attaccante che vuole sfruttare un bug di tipo OOB read per effettuare una information disclosure (come avvenuto per il celebre caso HeartBleed).

Risposta:

- **DEP / NX**: Politica del Processore che segnala o previene aree della Memoria marcate per *Data* di non essere mai potute incanalare per l'esecuzione di opcode CPU, respingendo un Code Injection malevolo (D'Elia, sec. 6.5 *DEP*, pag. 59).
- **Impatto su HeartBleed (OOB read, D'Elia sec. 4.3.4, pag. 32)**: DEP **NON** ostacola HeartBleed! Trattasi difatti di un *Information Disclosure exploit* fondato interamente sulla Memory Safety leakando memorie segrete limitrofe all'array, ma il payload non ha mai tentato *code execution*.

Quesito 9. *(se possibile, rispondere entro 10 righe)*
Definire la proprietà di spatial memory safety. Discutere bug types noti di questo tipo e se essi possono essere identificati da un sanitizer che usa redzone (se possibile, distinguere tra i casi di compiler-assisted vs. binary-level instrumentation per realizzare il sanitizer).

Risposta:

- **Spatial Memory Safety**: È un sistema dove un accesso ai cursori/puntatori referenzia categoricamente allocazioni limitrofe previste del medesimo oggetto (D'Elia, sec. 4.2.2 *Spatial safety*, pag. 27).
- **Bug types**: Buffer Overflow (esplorazione stack/heap in eccesso), Accessi *Out-Of-Bounds* r/w.
- **Redzone Analyzer**: AddressSanitizer (D'Elia, sec. 5.2.1 *Redzones*, pag. 39) posiziona padding letali di "redzone" contigui all'allocazione e si abbatte alla loro violazione. Modelli **compiler-assisted** (es. ASan in Clang) tracciano size precisi a zero-time (alte precisioni/prestazioni rispetto al binary-level come Valgrind).

3 Altre domande - Visconti

Qui vengono proposte altre domande sui temi del prof. Visconti, non sono veri appelli d'esame come i precedenti, ma simulazioni.

Domanda 1: One-Time Pad e Sicurezza Perfetta

Sia E uno schema di cifratura che utilizza One-Time Pad. Un avversario intercetta due ciphertext c_1 e c_2 generati con la stessa chiave k. Spiegare perché questo compromette la sicurezza del sistema e descrivere un attacco concreto che l'avversario può eseguire per recuperare informazioni sui messaggi originali.

Risposta Quando un avversario intercetta due ciphertext c_1 e c_2 generati con la stessa chiave k in uno schema One-Time Pad, la sicurezza del sistema viene completamente compromessa. Ecco perché:

- **Perché compromette la sicurezza** Il One-Time Pad garantisce sicurezza perfetta solo quando la chiave:
  1. È completamente casuale
  2. È lunga almeno quanto il messaggio
  3. Viene usata una sola volta (da qui il nome "One-Time")

  Quando la stessa chiave viene riutilizzata, la proprietà fondamentale del One-Time Pad viene violata: il ciphertext non è più indistinguibile da un output casuale e rivela informazioni sui messaggi originali.

- **Attacco concreto** Se un avversario intercetta due ciphertext $c_1 = m_1 \oplus k$ e $c_2 = m_2 \oplus k$ generati con la stessa chiave k, può eseguire il seguente attacco:
  1. Calcolare $c_1 \oplus c_2 = (m_1 \oplus k) \oplus (m_2 \oplus k) = m_1 \oplus m_2$

  Questo perché: $(m_1 \oplus k) \oplus (m_2 \oplus k) = m_1 \oplus k \oplus m_2 \oplus k = m_1 \oplus m_2 \oplus (k \oplus k) = m_1 \oplus m_2 \oplus 0 = m_1 \oplus m_2$

  Nota che $\oplus$ dello stesso elemento dà sempre come risultato 0. Ottenendo $m_1 \oplus m_2$, l'avversario ha eliminato la chiave k dall'equazione e ha ottenuto una relazione diretta tra i due messaggi originali.

- **Conseguenze dell'attacco**
  1. **Analisi statistica:** Conoscendo $m_1 \oplus m_2$, l'avversario può sfruttare le proprietà statistiche del linguaggio per dedurre informazioni sui messaggi originali.
  2. **Attacco a testo in chiaro noto:** Se l'avversario conosce o può indovinare parti di uno dei messaggi (ad esempio, intestazioni standard, formati prevedibili), può recuperare le parti corrispondenti dell'altro messaggio: Se conosce $m_1[i]$, può calcolare $m_2[i] = (m_1 \oplus m_2)[i] \oplus m_1[i]$
  3. **Attacco a testo in chiaro scelto:** Se l'avversario può influenzare il contenuto di uno dei messaggi, il problema diventa ancora più grave, potendo potenzialmente recuperare l'intero altro messaggio.

Questo dimostra perché il nome "One-Time Pad" enfatizza l'importanza cruciale di utilizzare la chiave una sola volta. Il riutilizzo della chiave trasforma un sistema con sicurezza perfetta in un sistema completamente vulnerabile.

Domanda 2: Modalità di Cifratura a Blocchi

Confrontare le modalità di cifratura ECB e CBC, evidenziando le differenze in termini di sicurezza. Fornire un esempio concreto di vulnerabilità presente in ECB ma non in CBC.

Risposta Confronto tra ECB e CBC:

- **Electronic Code Book (ECB)** La modalità Electronic Code Book (ECB) è la più semplice modalità operativa dei cifrari a blocchi:
  - **Funzionamento:** Ogni blocco del messaggio in chiaro viene cifrato indipendentemente usando la stessa funzione di cifratura con la stessa chiave.
  - **Caratteristica principale:** È una modalità deterministica - lo stesso blocco di input produce sempre lo stesso blocco cifrato.
  - **Formula:** $C_i = F_k(M_i)$ dove M_i è l'i-esimo blocco del messaggio e C_i è il corrispondente blocco cifrato.
- **Cipher Block Chaining (CBC)**
  - **Funzionamento:** Ogni blocco del messaggio in chiaro viene prima combinato (XOR) con il blocco cifrato precedente, e poi cifrato. Per il primo blocco, si utilizza un vettore di inizializzazione (IV) casuale.
  - **Caratteristica principale:** Introduce casualità nel processo di cifratura attraverso l'IV e la dipendenza tra blocchi.
  - **Formula:**
    $C_1 = F_k(M_1 \oplus IV)$
    $C_i = F_k(M_i \oplus C_{i-1})$ per $i > 1$
- **Differenze in termini di sicurezza**

Aspetto	**ECB**	**CBC**
Sicurezza CPA	Non è CPA-sicuro	È CPA-sicuro
Pattern nei dati	Preserva i pattern nei dati originali	Maschera i pattern grazie all'IV e al chaining
Blocchi identici	Blocchi identici nel messaggio producono blocchi identici nel testo cifrato	Blocchi identici nel messaggio producono blocchi diversi nel testo cifrato
Parallelizzazione	Può essere parallelizzato (cifratura simultanea di blocchi)	Richiede cifratura sequenziale
Resistenza agli attacchi	Vulnerabile ad analisi statistica e pattern recognition	Resistente a questi attacchi grazie alla casualità introdotta

- **Esempio concreto di vulnerabilità in ECB ma non in CBC** Un esempio emblematico della vulnerabilità di ECB è la cifratura di immagini. Quando un'immagine viene cifrata con ECB:
  1. Pixel identici vengono cifrati nello stesso modo, producendo pixel cifrati identici
  2. La struttura dell'immagine originale rimane visibile nel testo cifrato

  Questo è dimostrato dall'esempio del pinguino Linux (Tux) negli appunti, dove l'immagine cifrata con ECB mantiene chiaramente riconoscibile la forma del pinguino, nonostante la cifratura. Un avversario può facilmente dedurre informazioni sulla struttura dell'immagine originale senza conoscere la chiave.

CBC In CBC, questo problema non si verifica perché:

1. Anche se due blocchi (o pixel) sono identici nel messaggio originale, i loro blocchi cifrati saranno diversi grazie all'XOR con il blocco cifrato precedente
2. L'uso dell'IV garantisce che anche cifrando più volte la stessa immagine si ottengano risultati diversi

Un caso reale di questa vulnerabilità è stato Zoom, che utilizzava la modalità ECB fino al 2020, potenzialmente esponendo informazioni sensibili nelle comunicazioni cifrate. In conclusione, mentre ECB offre semplicità e parallelizzazione, le sue vulnerabilità in termini di sicurezza lo rendono inadatto per la maggior parte delle applicazioni moderne, dove CBC (o altre modalità come CTR) sono preferite per la loro maggiore sicurezza.

Domanda 3: Pseudorandom Generator

Sia G un generatore pseudocasuale che espande un seed di 128 bit in una stringa di 256 bit. Consideriamo la seguente costruzione per un nuovo generatore G': $G'(s) = G(s) \parallel G(G(s)[1...128])$ dove $\parallel$ indica la concatenazione e $G(s)[1...128]$ indica i primi 128 bit dell'output di $G(s)$. G' è un generatore pseudocasuale sicuro se G lo è? Giustificare la risposta.

Risposta La domanda chiede se $G'(s) = G(s) \parallel G(G(s)[1...128])$ è un generatore pseudocasuale sicuro quando G lo è.

- **Risposta**: No, G' **non** è un generatore pseudocasuale sicuro anche se G lo è, perché introduce una relazione deterministica e verificabile tra la prima e la seconda metà dell'output.
- **Distinguisher esplicito**: Dato un output candidato Y di 512 bit, un distinguisher efficiente agisce come segue:
  1. Prendi i primi 256 bit di Y, chiamali $X = Y[1 \dots 256]$.
  2. Calcola $G(X[1 \dots 128])$, ottenendo 256 bit.
  3. Verifica se $Y[257 \dots 512] = G(X[1 \dots 128])$.
     - Se Y è stato prodotto da G', la condizione vale **sempre** (probabilità 1).
     - Se Y è una stringa casuale vera, la condizione vale con probabilità 2^{-256}, quindi trascurabile.

  Il vantaggio del distinguisher è quindi ≈ 1, demolendo la sicurezza pseudocasuale di G'.
- **Regola pratica**:
  - **OK**: se i due blocchi provengono da generatori con seed **indipendenti**, la concatenazione resta sicura.
  - **NON OK**: se la seconda parte è funzione deterministica della prima (come in G'), si introduce una correlazione verificabile in tempo polinomiale e si perde la sicurezza pseudocasuale.

Domanda 4: Diffie-Hellman e Man-in-the-Middle

Alice e Bob utilizzano il protocollo Diffie-Hellman per stabilire una chiave condivisa. Descrivere in dettaglio come un attaccante può eseguire un attacco Man-in-the-Middle e quali contromisure possono essere adottate per prevenirlo.

Risposta

- **Dinamica dell'attacco** Nel protocollo Diffie-Hellman, un attacco Man-in-the-Middle (MITM) può essere eseguito nel seguente modo:
  1. L'attaccante si posiziona tra Alice e Bob intercettando la loro comunicazione
  2. Quando Alice invia il suo valore pubblico g^a, l'attaccante lo intercetta e invia a Bob un proprio valore g^s
  3. Similmente, quando Bob invia il suo valore pubblico g^b, l'attaccante lo intercetta e invia ad Alice un valore g^t
  4. Come risultato:

  - Alice calcola una chiave $K = g^{at}$ con l'attaccante
  - Bob calcola una chiave $K' = g^{bs}$ con l'attaccante

  L'attaccante può ora decifrare e manipolare tutti i messaggi tra Alice e Bob.

- **Contromisure principali**
  - Utilizzare certificati digitali per autenticare le identità dei partecipanti
  - Implementare firme digitali per verificare l'autenticità dei valori scambiati
  - Utilizzare una Certificate Authority (CA) affidabile per validare le identità

  Queste misure permettono di verificare l'autenticità dei partecipanti e prevenire attacchi MITM.

Domanda 5: RSA e Padding (non spiegato nel dettaglio)

Spiegare perché l'implementazione "textbook" di RSA (senza padding) non è sicura per la cifratura di messaggi. Descrivere almeno due attacchi possibili e come il padding OAEP risolve questi problemi.

Risposta

- **Vulnerabilità dell'implementazione "textbook"** L'implementazione "textbook" di RSA senza padding presenta diverse vulnerabilità critiche:
  1. **Attacco di malleabilità:** A causa della proprietà moltiplicativa di RSA, un attaccante può modificare un ciphertext $c = m^e \pmod n$ moltiplicandolo per $(2^e \pmod n)$, ottenendo un nuovo ciphertext valido che decrittato darà $2m$. Questo permette di manipolare il messaggio cifrato senza conoscere il contenuto.
  2. **Attacco deterministico:** RSA senza padding è deterministico, quindi lo stesso messaggio produce sempre lo stesso ciphertext. Un attaccante può creare una tabella di messaggi comuni e i loro ciphertext per decifrare messaggi intercettati.
- **Soluzione tramite Padding OAEP** Il padding OAEP (Optimal Asymmetric Encryption Padding) risolve questi problemi:
  - Aggiunge casualità al messaggio prima della cifratura, rendendo la cifratura probabilistica
  - Impedisce la malleabilità del ciphertext grazie alla struttura del padding
  - Limita la dimensione del messaggio a una lunghezza fissa, prevenendo attacchi basati sulla lunghezza

  Inoltre, OAEP garantisce la sicurezza semantica contro attacchi CPA (Chosen Plaintext Attack) e CCA (Chosen Ciphertext Attack).

Domanda 6: Collision Resistance

Sia $H : \{0,1\}^* \to \{0,1\}^{160}$ una funzione hash. Considerando il paradosso del compleanno, quanti messaggi dovrebbero essere generati per avere una probabilità del 50% di trovare una collisione? Spiegare perché questo risultato è rilevante per la sicurezza delle firme digitali.

Risposta

- **Probabilità di collisione** Per trovare una collisione con probabilità del 50% in una funzione hash con output di 160 bit, servono circa 2^{80} messaggi. Questo deriva dal paradosso del compleanno: per uno spazio di output di dimensione 2^n, la probabilità del 50% di collisione si raggiunge con circa $\sqrt{2^n} = 2^{n/2}$ tentativi. Con $n = 160$, otteniamo 2^{80}.

- **Rilevanza per le firme digitali**
  - Le collisioni permettono attacchi di falsificazione: se un attaccante trova due messaggi M_1 e M_2 con $H(M_1) = H(M_2)$, può far firmare M_1 (legittimo) e poi sostituirlo con M_2 (malevolo) mantenendo la stessa firma valida
  - Questo compromette l'integrità e l'autenticità della firma digitale

  Per questo motivo si preferiscono hash con almeno 256 bit (come SHA-256), che richiederebbero 2^{128} operazioni per trovare collisioni.

(Nota: Domanda simile a quella fatta dal prof)

Domanda 7: Merkle-Damgård Construction (non spiegato nel dettaglio)

Descrivere la costruzione di Merkle-Damgård utilizzata in molte funzioni hash come SHA-256. Spiegare come questa costruzione garantisce la resistenza alle collisioni se la funzione di compressione sottostante è resistente alle collisioni.

Risposta La costruzione di Merkle-Damgård processa messaggi di lunghezza arbitraria dividendoli in blocchi di dimensione fissa.

- **Struttura**
  1. **Padding:** il messaggio viene esteso con bit di padding e la sua lunghezza originale
  2. **Divisione:** il messaggio viene diviso in blocchi di dimensione fissa
  3. **Iterazione:** ogni blocco viene processato sequenzialmente da una funzione di compressione $f(H_{i-1}, M_i)$ dove H_{i-1} è lo stato precedente e M_i il blocco corrente
  4. **Concatenazione:** H_0 (valore iniziale) $\rightarrow H_1 \rightarrow H_2 \rightarrow ... \rightarrow H_n$ (hash finale)
- **Garanzia di sicurezza** Se la funzione di compressione è collision-resistant, allora l'intera costruzione è collision-resistant. Dimostrazione (per contraddizione):
  - Se esistesse una collisione $H(M) = H(M')$ per messaggi diversi $M \neq M'$
  - Dopo il padding e la divisione in blocchi, i messaggi processati sarebbero diversi
  - Tracciando all'indietro la catena di compressioni, dovrebbe esistere almeno un punto dove $H_i = H'_i$ ma $(H_{i-1}, M_i) \neq (H'_{i-1}, M'_i)$
  - Questo violerebbe la collision-resistance della funzione di compressione

  Questa proprietà di riduzione della sicurezza rende la costruzione molto robusta.

Domanda 8: Length Extension Attack

Sia H una funzione hash basata sulla costruzione di Merkle-Damgård. Spiegare in cosa consiste un length extension attack e perché HMAC è resistente a questo tipo di attacco mentre $H(k \parallel m)$ non lo è.

Risposta Un Length Extension Attack sfrutta la struttura iterativa di Merkle-Damgård per estendere un hash senza conoscere il messaggio originale. In questo schema, SHA-256 processa iterativamente i blocchi del messaggio, mantenendo uno stato interno che viene aggiornato progressivamente.

- **Come funziona l'attacco**
  - Dato $H(k \parallel m)$ e conoscendo la lunghezza di $k \parallel m$, un attaccante può calcolare $H(k \parallel m \parallel \text{padding} \parallel m')$ per un messaggio m' arbitrario
  - Questo perché può utilizzare $H(k \parallel m)$ come stato interno per continuare l'hashing

  - Non serve conoscere k o m, solo il loro hash e la lunghezza totale

- **Perché $H(k \parallel m)$ è vulnerabile** L'hash finale diventa lo stato interno utilizzabile per estendere il messaggio.

- **Perché HMAC è resistente**

  - HMAC usa la costruzione: $H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m))$
  - L'hash interno $H((k \oplus \text{ipad}) \parallel m)$ viene re-hashato con una chiave diversa
  - Anche se si potesse estendere l'hash interno, servirebbe conoscere ($k \oplus$opad) per calcolare l'hash esterno finale
  - La doppia applicazione con chiavi diverse impedisce l'estensione.

Conseguenza: per l'autenticazione di messaggi, usare sempre HMAC invece di $H(k \parallel m)$. (vedere pagina 24)

Domanda 9: HMAC vs CBC-MAC (non spiegata nel dettaglio)

Confrontare HMAC e CBC-MAC in termini di sicurezza e applicabilità. In quali scenari è preferibile utilizzare HMAC rispetto a CBC-MAC e perché?

Risposta HMAC è più sicuro e versatile di CBC-MAC perché resiste meglio agli attacchi anche se il messaggio ha lunghezza variabile, grazie all'uso di funzioni hash crittografiche. CBC-MAC è sicuro solo per messaggi di lunghezza fissa. Preferibile usare HMAC in protocolli generici (es. HTTPS, API auth) per la sua robustezza e semplicità d'uso su messaggi di lunghezza variabile.

- **Quando preferire HMAC**

  - Messaggi di lunghezza variabile - CBC-MAC standard è vulnerabile
  - Semplicità implementativa - meno prone ad errori
  - Quando si usano già funzioni hash nel sistema
  - Maggiore robustezza contro implementazioni scorrette
  - Standard più diffuso per autenticazione web/network

- **Quando preferire CBC-MAC**

  - Sistemi embedded dove si usa già un cifrario a blocchi
  - Messaggi sempre di lunghezza fissa
  - Prestazioni critiche con hardware dedicato per AES

Conclusione: HMAC è generalmente preferibile per la maggior parte delle applicazioni grazie alla sua robustezza e versatilità.

Domanda 10: Authenticated Encryption

Spiegare cosa si intende per "authenticated encryption" e perché è importante. Descrivere la differenza tra gli approcci Encrypt-then-MAC, MAC-then-Encrypt e Encrypt-and-MAC, indicando quale dei tre è generalmente considerato più sicuro e perché.

Risposta

- **Authenticated Encryption e Importanza** L'authenticated encryption combina crittografia e autenticazione in un unico processo, garantendo sia la confidenzialità (i dati sono cifrati) che l'integrità/autenticità (i dati non sono stati modificati). Previene attacchi come padding oracle, bit-flipping e replay attacks che potrebbero compromettere sistemi che usano solo crittografia senza autenticazione.
- **Tre approcci principali**
  1. **Encrypt-then-MAC:** Prima cifra il messaggio, poi calcola il MAC sul ciphertext ($C = \text{Encrypt}(M)$, $T = \text{MAC}(C)$)
  2. **MAC-then-Encrypt:** Prima calcola il MAC, poi cifra messaggio + MAC ($T = \text{MAC}(M)$, $C = \text{Encrypt}(M \parallel T)$)
  3. **Encrypt-and-MAC:** Cifra il messaggio e calcola il MAC sul plaintext in parallelo ($C = \text{Encrypt}(M)$, $T = \text{MAC}(M)$)
- **Approccio più sicuro** Encrypt-then-MAC è considerato il più sicuro perché:
  - Il MAC protegge l'intero ciphertext da modifiche
  - Non rivela informazioni sul plaintext attraverso il MAC
  - Permette di verificare l'integrità prima della decifratura

  È l'approccio usato in standard moderni come AES-GCM e ChaCha20-Poly1305. Gli altri due approcci presentano vulnerabilità teoriche e pratiche in determinate implementazioni.

(pagina 20)

Domanda 11: TLS Handshake

Descrivere le fasi principali del TLS 1.3 handshake, evidenziando le differenze rispetto a TLS 1.2. Spiegare come TLS 1.3 migliora la sicurezza e riduce la latenza.

Risposta

- **Fasi principali TLS 1.3**
  1. **ClientHello:** Client invia cipher suites supportate + key shares pre-calcolate. Include estensioni e parametri crittografici.
  2. **ServerHello + dati cifrati:** Server sceglie parametri e invia la sua key share. Immediatamente invia Certificate, CertificateVerify, Finished (già cifrati).
  3. **Client Finished:** Client verifica certificato e invia Finished cifrato. Handshake completato in 1-RTT.
- **Differenze da TLS 1.2**
  - TLS 1.2: 2-RTT (4 messaggi di andata/ritorno)
  - TLS 1.3: 1-RTT (2 messaggi di andata/ritorno)
  - Pre-condivisione chiavi: Client invia key shares nel primo messaggio
  - Always encrypted: Tutto dopo ServerHello è cifrato
  - 0-RTT opzionale: Per connessioni successive con PSK
- **Miglioramenti**
  - **Sicurezza:** Eliminati algoritmi deboli (RSA key exchange, DES, RC4, MD5, SHA-1). Perfect Forward Secrecy obbligatoria. Certificati sempre cifrati (privacy).
  - **Latenza:** 50% riduzione (da 2-RTT a 1-RTT). 0-RTT per sessioni riprese (con trade-off di sicurezza). Handshake più semplice e lineare.

(pagina 48)

Domanda 12: Forward Secrecy

Spiegare il concetto di "forward secrecy" nel contesto di TLS. Perché è importante e come viene implementato in TLS 1.3?

Risposta

- **Concetto e Importanza** La forward secrecy (o Perfect Forward Secrecy - PFS) garantisce che la compromissione delle chiavi private del server non permetta di decifrare le comunicazioni passate già intercettate. È importante per:
  - Protezione retroattiva: Anche se un attaccante ottiene le chiavi private del server, non può decifrare traffico precedentemente catturato
  - Sicurezza a lungo termine: Essenziale contro attacchi "harvest now, decrypt later"
  - Compliance: Richiesto da molti standard di sicurezza
- **Come funziona** Invece di usare le chiavi private del server per cifrare direttamente, si generano chiavi di sessione temporanee che vengono eliminate dopo l'uso.
- **Implementazione in TLS 1.3**
  1. **Ephemeral Key Exchange obbligatorio:** Solo algoritmi come ECDHE e DHE.
  2. **Eliminazione RSA key exchange:** TLS 1.2 permetteva RSA statico (no forward secrecy), TLS 1.3 lo rimuove completamente.
  3. **Processo:** Client e server generano coppie di chiavi temporanee, scambiano le chiavi pubbliche temporanee, derivano chiavi di sessione condivise e infine eliminano le chiavi temporanee private dopo l'handshake.
  4. **Perfect Forward Secrecy sempre attiva:** Non è opzionale come in TLS 1.2, ogni sessione usa nuove chiavi temporanee.

(pagina 47)

Domanda 16: Algoritmo di Shor (non spiegato)

Spiegare come l'algoritmo di Shor minaccia la sicurezza dei sistemi crittografici asimmetrici. Quali algoritmi di crittografia asimmetrica sono vulnerabili e quali sono considerati resistenti agli attacchi quantistici?

Risposta Ad esempio, l'algoritmo di Shor può fattorizzare numeri interi in modo efficiente, mettendo a rischio la sicurezza dei sistemi di crittografia asimmetrica (come RSA e Diffie-Hellman). (pagina 6)

Domanda 17: Algoritmo di Grover (non spiegato)

Descrivere l'impatto dell'algoritmo di Grover sulla sicurezza della crittografia simmetrica. Come dovrebbero essere adattate le dimensioni delle chiavi per mantenere lo stesso livello di sicurezza in presenza di computer quantistici?

Risposta Per quanto riguarda la crittografia simmetrica, i computer quantistici non offrono una potenza di calcolo superiore in senso classico, ma consentono uno speed-up degli attacchi brute force grazie all'algoritmo di Grover. Questo algoritmo riduce il numero di tentativi necessari passando da $O(2^n)$ a $O(2^{n/2})$. Di conseguenza, una chiave a 128 bit, che in un attacco classico richiederebbe 2^{128} tentativi, avrebbe una sicurezza equivalente a una chiave a circa 64 bit in presenza di attacchi quantistici. Per questo motivo, è necessario considerare chiavi di lunghezza maggiore o sviluppare nuove tecniche crittografiche resistenti agli attacchi quantistici. (pagina 6)

Domanda 18: Onion Routing

Descrivere il funzionamento dell'Onion Routing e come garantisce l'anonimato degli utenti. Quali sono i limiti di questa tecnologia e quali attacchi possono comprometterla?

Risposta

- **Funzionamento**
  1. **Architettura a strati:** Il messaggio viene cifrato più volte con le chiavi pubbliche di relay intermedi. Ogni relay può decifrare solo il suo "strato" esterno. Come una cipolla: ogni strato nasconde quello interno. Ogni server conosce solo il suo predecessore e il suo successore.
  2. **Processo:** Client sceglie un circuito di 3+ relay (guard, middle, exit).
     Cifra: $E_\text{exit}(E_\text{middle}(E_\text{guard}(\text{messaggio})))$. Ogni relay decifra il suo strato e inoltra al successivo. Solo l'exit node vede il contenuto finale.
- **Garanzie di anonimato**
  - **Unlinkability:** Nessun singolo relay conosce sia mittente che destinazione. Guard node: conosce client ma non destinazione. Exit node: conosce destinazione ma non client originale.
  - **Traffic analysis resistance:** Padding e timing uniforme. Routing attraverso relay distribuiti globalmente.
- **Limiti principali**
  - **Exit node vulnerabilities:** Exit node vede traffico in chiaro (se non HTTPS). Possibile monitoraggio o manipolazione.
  - **Latenza elevata:** Routing attraverso 3+ hop aumenta significativamente i tempi.
  - **Bandwidth limitato:** Capacità della rete dipende dai volontari.
- **Attacchi principali**
  - **Traffic Analysis:** Correlation attacks, Website fingerprinting.
  - **Exit node attacks:** Malicious exit nodes, SSL stripping.
  - **Guard node compromise:** Se compromesso a lungo termine, può de-anonimizzare utenti.
  - **Global adversary:** Chi controlla gran parte della rete può correlare traffico end-to-end.
  - **Side-channel attacks:** Timing attacks, analisi consumo energetico, browser fingerprinting.

Domanda 20: PBKDF2 e Password Hashing

Spiegare perché è importante utilizzare funzioni come PBKDF2 per l'archiviazione delle password invece di semplici hash. Quali parametri influenzano la sicurezza di PBKDF2 e come dovrebbero essere scelti?

Risposta

- **Perché PBKDF2 invece di semplici hash** Per cifrare i dati su disco, si utilizzano algoritmi di crittografia simmetrica come AES. È importante notare che funzioni hash come SHA-256 non sono algoritmi di cifratura e non possono essere utilizzate per decifrare dati. PBKDF2 introduce un costo computazionale deliberato che rende gli attacchi molto più lenti e costosi.
- **Parametri chiave di PBKDF2 e come sceglierli**

1. **Salt:** Valore casuale unico per ogni password che previene attacchi rainbow table. Deve essere di almeno 128 bit e generato crittograficamente.
2. **Iterazioni:** Numero di cicli di hash (tipicamente 100.000-600.000 per PBKDF2-SHA256). Bisogna bilanciare sicurezza e performance (target: 100ms di calcolo) e aggiornarle periodicamente seguendo l'evoluzione hardware.
3. **Lunghezza della chiave:** Dimensione dell'output finale (almeno 256 bit).
4. **Funzione hash:** Algoritmo sottostante (SHA-256 o SHA-512).

(pagina 30)

Domanda 21: OpenSSL e Comandi di Base

I seguenti comandi OpenSSL sono utilizzati per generare e verificare firme digitali:

```
openssl dgst -sha256 -sign private_key.pem -out signature.bin document.txt
openssl dgst -sha256 -verify public_key.pem -signature signature.bin document.txt
```

Spiegare cosa fa ciascun comando e come funziona il processo di firma e verifica. Quali algoritmi di hash e firma vengono utilizzati in questo esempio?

Risposta

- **Primo comando - Generazione della firma** Calcola l'hash SHA-256 del file `document.txt`, firma l'hash usando la chiave privata contenuta in `private_key.pem` e salva la firma digitale nel file `signature.bin`.
- **Secondo comando - Verifica della firma** Calcola l'hash SHA-256 del file `document.txt`, decifra la firma usando la chiave pubblica in `public_key.pem`, confronta l'hash calcolato con quello decifrato dalla firma e restituisce "Verified OK" se corrispondono.
- **Processo completo**
  1. **Firma:** Hash del documento → Cifratura con chiave privata → Firma digitale.
  2. **Verifica:** Hash del documento + Decifratura firma con chiave pubblica → Confronto.
- **Algoritmi utilizzati** Hash: SHA-256 (specificato con `-sha256`). Firma: RSA (default di OpenSSL, determinato dal tipo di chiave). La sicurezza dipende da entrambi: SHA-256 per l'integrità, RSA per l'autenticazione.

(pagina 61)

Domanda 22: Certificati X.509

Descrivere la struttura di un certificato X.509v3 e il ruolo delle estensioni. In particolare, spiegare l'importanza dell'estensione "Subject Alternative Name" nel contesto delle connessioni HTTPS.

Risposta

- **Struttura principale (3 parti)**
  1. **TBSCertificate ("To Be Signed"):** Contiene Versione (v3), Numero di serie, Algoritmo di firma, Emittente (Issuer), Periodo di validità, Soggetto (Subject), Chiave pubblica del soggetto ed Estensioni.
  2. **Algoritmo di firma:** Specifica l'hash e lo schema di firma (es. SHA-256 con RSA).
  3. **Firma:** Il valore crittografico calcolato su TBSCertificate con la chiave privata dell'Issuer.

- **Ruolo delle estensioni (v3)** Le estensioni permettono di aggiungere attributi opzionali o vincoli. Possono essere *Critical* (deve essere rispettata dal client) o *Non-critical* (informativa). Esempi: Basic Constraints, Key Usage, Extended Key Usage, Certificate Policies.
- **Subject Alternative Name (SAN) in HTTPS** Estensione non-critical che elenca nomi aggiuntivi (DNS, IP, URI) per cui il certificato è valido. I browser controllano il SAN anziché il campo Subject CN. Consentendo più nomi (es. dominio, www, sottodomini, IP), evita errori di "nome host non corrispondente" e supporta certificati multi-domain o wildcard. Garantisce che la connessione TLS sia stabilita solo se l'hostname richiesto compare nel SAN, prevenendo attacchi MITM.

(pagina 45)

Domanda 23: Side-Channel Attacks

Descrivere cosa sono gli attacchi side-channel e fornire almeno tre esempi. Quali contromisure possono essere adottate per mitigare questi attacchi?

Risposta Gli attacchi side-channel sfruttano informazioni "laterali" derivanti dall'implementazione fisica di un sistema crittografico, non dall'algoritmo stesso. Sono particolarmente pericolosi perché aggirano la sicurezza matematica degli algoritmi.

- **Tre esempi principali**
  - **Temperature attacks:** misurazione delle variazioni di temperatura durante l'esecuzione di operazioni crittografiche
  - **Power analysis:** misurazione del consumo energetico durante l'esecuzione
  - **Timing attacks:** misurazione del tempo richiesto per eseguire operazioni di sicurezza
- **Contromisure**
  - **Constant-time algorithms:** Operazioni con tempo fisso indipendente dai dati
  - **Masking/Blinding:** Randomizzazione dei dati intermedi
  - **Noise injection:** Aggiunta di rumore per mascherare i segnali
  - **Secure coding practices:** Evitare confronti variabili e operazioni data-dependent

(pagina 67)

Domanda 24: Replay Attack

Spiegare cosa si intende per "replay attack" e come può essere prevenuto nei protocolli di autenticazione. Fornire un esempio concreto di protocollo vulnerabile a questo tipo di attacco e la relativa correzione.

Risposta

- **Cos'è un Replay Attack** Il replay attack si verifica quando un attaccante intercetta e riutilizza una comunicazione valida (come un messaggio o una transazione) per ottenere un risultato non autorizzato.
- **Prevenzione** Questo tipo di attacco può essere mitigato tramite l'uso di tecniche di sincronizzazione e timestamp, che rendono ogni comunicazione unica e non riutilizzabile. I timestamp sono informazioni temporali aggiunte ai messaggi per garantirne l'unicità e la validità nel tempo.

(pagina 20, 66)

4 Altre domande - D'Elia

Domanda 1: Fuzzing e Feedback

Spiegare il ruolo del feedback di copertura (feedback di coverage) nel fuzzing. Quali sono i vantaggi dell'edge coverage rispetto ad altre metriche di copertura e come viene implementata in AFL?

Risposta

- **Ruolo del feedback di copertura** Il feedback di copertura nel fuzzing guida l'esplorazione del programma, permettendo al fuzzer di identificare input che esercitano nuovi percorsi e di generare test sempre più efficaci.
- **Vantaggi dell'edge coverage ed implementazione in AFL** L'edge coverage è la metrica più usata perché bilancia sensibilità e overhead: distingue tra percorsi diversi senza generare troppi input ridondanti e mantiene prestazioni elevate. In AFL, l'edge coverage si implementa tramite una mappa di copertura che traccia le transizioni tra blocchi di codice (edge) usando hash o identificatori univoci. Questo consente di individuare rapidamente nuovi comportamenti e di mantenere una coda di input "interessanti", ottimizzando la scoperta di bug.

(pagina 74)

Domanda 2: Sanitizers - MSan vs ASan

Confrontare MemorySanitizer (MSan) e AddressSanitizer (ASan) in termini di tipi di errori rilevati, funzionamento interno e limitazioni.

Risposta

1. **Tipi di errori rilevati**
   - **MSan**: Rileva l'uso di memoria non inizializzata (UUM). Non segnala la sola lettura di byte non inizializzati, ma traccia la *propagazione* del taint: l'errore scatta solo quando il valore non inizializzato viene effettivamente **usato** in una computazione significativa (condizioni di salto, argomenti di syscall, operazioni aritmetiche che influenzano il flusso).
   - **ASan**: Si focalizza sulla spatial memory safety (buffer overflow/underflow, accessi out-of-bounds heap/stack/global) e copre parzialmente la temporal safety (use-after-free, use-after-return, use-after-scope).
2. **Funzionamento interno**
   - **MSan**: Utilizza una shadow memory con mappatura 1:1 per ogni byte di memoria del programma. Strumenta **ogni istruzione** (non solo load/store come ASan) per propagare il taint attraverso assegnamenti, operazioni aritmetiche e logiche: se un operando è "tainted" (non inizializzato), il risultato lo diventa a sua volta (*shadow propagation*). L'errore viene segnalato nel punto in cui il valore tainted viene consumato in modo significativo. Richiede che tutte le librerie siano ricompilate con MSan.
   - **ASan**: Inserisce *redzones* attorno agli oggetti allocati e mantiene una shadow memory per lo stato di validità. Strumenta ogni operazione di load/store con un controllo `isPoisoned()`. Usa una quarantena per i blocchi liberati per rilevare use-after-free.
3. **Limitazioni**
   - **MSan**: Disponibile solo per LLVM/Clang. Richiede ricompilazione totale di tutte le dipendenze. Ha un overhead significativo ($\sim$3x) e non coesiste con ASan.
   - **ASan**: Approccio probabilistico per gli OOB (le redzones possono essere scavalcate da accessi molto distanti). Non rileva OOB intra-oggetto (tra campi della stessa struct). La quarantena per il temporal safety ha efficacia limitata nel tempo.

Domanda 3: Sicurezza della memoria temporale

Spiegare cosa si intende per "sicurezza della memoria temporale" e descrivere come AddressSanitizer rileva errori di tipo use-after-free, use-after-return e use-after-scope.

Risposta La sicurezza della memoria temporale garantisce che la memoria non venga usata dopo essere stata liberata o uscita dal proprio ambito. AddressSanitizer rileva:

- **Use-after-free:** mette in quarantena la memoria liberata e la marca come "avvelenata"; se viene acceduta, segnala l'errore.
- **Use-after-return:** protegge le variabili locali spostandole in heap e marcando lo stack come non valido dopo il ritorno dalla funzione.
- **Use-after-scope:** marca la memoria di variabili uscite dallo scope come avvelenata, segnalando accessi illegali.

Domanda 4: Threat Modeling

Descrivere il concetto di "threat modeling" e spiegare perché è importante nella progettazione di sistemi sicuri. Fornire esempi di almeno due diversi modelli di minaccia discussi nel corso.

Risposta

- **Concetto e Importanza** Il threat model è il processo di identificazione delle risorse da proteggere, dei vettori di minaccia e delle capacità dell'avversario, al fine di progettare contromisure adeguate già nelle fasi iniziali dello sviluppo (D'Elia, sec. 2.3 *Modello di Minaccia*). Un modello inadeguato porta direttamente a difese insufficienti, poiché ogni assunzione errata rappresenta una potenziale vulnerabilità.
- **Modelli di minaccia trattati nel corso**
  1. **Utente di rete** (D'Elia, sec. 2.3.1): Attaccante anonimo che interagisce con il servizio tramite rete. Può misurare tempi e dimensioni delle risposte, inviare input malformati, eseguire sessioni parallele. Esempi di attacchi: SQL injection, buffer overflow remoto, ROP payload.
  2. **Utente snooping** (D'Elia, sec. 2.3.2): Attaccante sulla stessa rete delle vittime (es. Wi-Fi aperto). Può leggere, intercettare e modificare i messaggi altrui. Esempi di attacchi: session hijacking, attacchi DoS, violazioni di privacy via side-channel.
  3. **Utente co-locato** (D'Elia, sec. 2.3.3): Attaccante che esegue codice sulla stessa macchina della vittima (es. malware installato). Può leggere file, memoria e sequenze di tasto. Esempio di attacco: furto di credenziali salvate in memoria.

Domanda 5: Analisi dei crash nel fuzzing

Descrivere il processo di analisi dei crash in un fuzzer come AFL/AFL++. Come vengono raggruppati i crash e quali sfide si incontrano nel determinare il numero effettivo di bug unici?

Risposta

- **Processo di analisi e deduplicazione** Nel fuzzing, l'analisi dei crash consiste nel raccogliere e classificare i casi anomali. I crash vengono raggruppati tramite tecniche di deduplicazione, come lo stack hashing, che confronta le stack trace per identificare crash simili basandosi sui primi k frame.
- **Sfide nel determinare i bug unici** Il parametro k è problematico: un k troppo basso unisce crash distinti, un k troppo alto separa crash dello stesso bug. Crash simili possono derivare da bug diversi, e bug identici produrre stack trace differenti. La deduplicazione evita analisi ridondanti ma porta facilmente a sotto/sovrastime. Spesso è richiesto triage manuale o strumenti avanzati.

Domanda 6: Undefined Behavior

Considerare il seguente frammento di codice C:

```
int a = 10;
int b = 0;
int c = a / b;
```

Identificare il tipo di undefined behavior presente, spiegare come UBSan lo rileva e quali potrebbero essere le conseguenze di questo comportamento in un programma reale.

Risposta

- **Undefined Behavior presente:** Il codice presenta un undefined behavior dovuto alla divisione intera per zero.
- **Rilevamento tramite UBSan:** In C, dividere un intero per zero non è definito. UndefinedBehaviorSanitizer inserisce controlli a runtime che intercettano l'operazione e segnalano l'errore con un messaggio dettagliato.
- **Conseguenze reali:** Può portare a crash immediati, corruzione di dati o vulnerabilità di sicurezza sfruttabili, rendendo il software inaffidabile.

Domanda 7: Use of Uninitialized Memory (UUM)

Considerare il seguente frammento di codice C:

```
int x;
int y = x + 5;
printf("%d\n", y);
```

Spiegare perché questo codice contiene un errore di tipo UUM, come MemorySanitizer (MSan) lo rileva e quali problemi di sicurezza può causare in applicazioni reali.

Risposta

- **Errore di tipo UUM:** Lo spazio per l'intero x viene allocato ma il suo valore è indefinito (può essere qualsiasi cosa nello stack). Usare il valore non inizializzato nell'espressione $y = x + 5$ genera l'UUM.
- **Rilevamento tramite MSan:** MSan traccia lo stato di inizializzazione di ogni variabile byte per byte; quando rileva l'uso di un valore "non inizializzato" in un'operazione, segnala l'errore a runtime.
- **Problemi di sicurezza reali:** L'uso di memoria non inizializzata può causare comportamenti imprevedibili, leak di informazioni sensibili, o corruzione di dati sfruttabile dagli attaccanti.

Domanda 8: Redzones in AddressSanitizer

Spiegare il concetto di "redzone" in AddressSanitizer, descrivendo come vengono implementate, quali valori di shadow memory sono associati ad esse e come contribuiscono al rilevamento di buffer overflow.

Risposta

- **Concetto e Implementazione** Le "redzone" in AddressSanitizer sono aree di memoria speciali inserite attorno agli oggetti allocati (heap, stack, globali) per proteggerne i confini. Queste zone non devono mai essere lette o scritte dal programma.
- **Shadow memory e valori** AddressSanitizer usa una shadow memory che mappa la memoria reale. Assegna valori specifici alle redzone (es. `0xF1` per heap redzone, `0xFA` per stack left redzone).
- **Rilevamento Buffer Overflow** Quando avviene un accesso in una redzone, la shadow memory lo identifica immediatamente come una violazione, generando l'errore. Questo facilita enormemente il debug e la sicurezza limitando i buffer overflow e underflow.

Domanda 9: Principi di sicurezza

Descrivere la triade CIA (Confidentiality, Integrity, Availability) e spiegare perché è considerata un modello parzialmente datato. Quali altre proprietà di sicurezza sono importanti nei sistemi moderni?

Risposta

- **La triade CIA e i suoi limiti** La triade CIA rappresenta i tre principi fondamentali: Confidentiality (riservatezza), Integrity (integrità) e Availability (disponibilità). È considerata parzialmente datata perché troppo ristretta rispetto alla complessità attuale dei sistemi.
- **Proprietà moderne aggiuntive** Oggi sono essenziali proprietà come privacy, non ripudiabilità, autenticazione, tracciabilità, resilienza agli attacchi, gestione dell'identità e protezione dei dati cloud. L'evoluzione delle minacce richiede questo approccio più ampio.

Domanda 10: Compartmentalizzazione

Spiegare il concetto di compartmentalizzazione nella sicurezza del software, come si relaziona con i principi di isolamento e minimo privilegio, e fornire un esempio concreto di applicazione (come i browser web moderni).

Risposta

- **Concetto di Compartmentalizzazione** Consiste nel suddividere un sistema in componenti isolati che interagiscono solo tramite interfacce controllate, limitando l'impatto di eventuali compromissioni.
- **Relazione con altri principi** È strettamente legata all'isolamento (separare le parti per evitare la propagazione degli attacchi) e al minimo privilegio (ogni compartimento ha solo i permessi strettamente necessari).
- **Esempio: Browser web** I browser moderni eseguono ogni scheda (tab) in un processo separato: se una scheda viene compromessa, l'attaccante non può accedere direttamente ai dati delle altre schede o infettare il sistema operativo.

Domanda 11: Fuzzing Harness

Descrivere cosa è un fuzzing harness, quali sono i suoi componenti principali e come viene implementato in pratica. Fornire un esempio di harness per una libreria come libxml2.

Risposta

- **Cosa è un fuzzing harness** È un piccolo programma o funzione che collega un fuzzer al codice da testare, permettendo di fornire input generati automaticamente e osservarne il comportamento. (`seed` → `fuzzer` → `harness` → `codice da testare`).
- **Componenti principali**
  1. Funzione di ingresso (entry point) che riceve l'input
  2. Preparazione dell'ambiente (inizializzazione e pulizia)
  3. Chiamata alle API della libreria da testare
- **Esempio libxml2** Un harness per `libxml2` è tipicamente una funzione che riceve un buffer generato dal fuzzer, lo passa a `xmlReadMemory` per eseguire il parsing e infine libera correttamente le risorse allocate per evitare memory leak durante il fuzzing.

(pagina 53)

Domanda 12: Metriche di performance nel fuzzing

Discutere le principali metriche utilizzate per valutare le prestazioni di un fuzzer (throughput, dimensione della coda, copertura) e come interpretarle correttamente per ottimizzare una campagna di fuzzing.

Risposta Le principali metriche di performance nel fuzzing sono:

- **Throughput:** numero di test case eseguiti al secondo; indica la velocità del fuzzer, ma non sempre la sua efficacia.
- **Dimensione della coda:** numero di input "interessanti" mantenuti; una coda troppo grande può rallentare il fuzzing, una troppo piccola può ridurre la copertura.
- **Copertura:** misura le parti di codice raggiunte; maggiore copertura indica una migliore esplorazione del programma.

Per ottimizzare una campagna di fuzzing, è importante bilanciare queste metriche: aumentare la copertura e mantenere un throughput elevato, evitando code troppo grandi che rallentano l'esecuzione. (pagina 76)

Domanda 13: Threat-driven design

Spiegare il concetto di "threat-driven design" e come le assunzioni errate nel modello di minaccia possono portare a vulnerabilità. Fornire un esempio concreto di assunzione errata e le sue conseguenze.

Risposta

- **Concetto** Il "threat-driven design" è un approccio alla progettazione che parte dall'analisi delle minacce: si identificano i possibili attaccanti, le loro capacità e gli obiettivi, e si progettano le difese di conseguenza.
- **Assunzioni errate ed Esempi** Sottovalutare una minaccia o fare assunzioni sbagliate porta a vulnerabilità. Ad esempio, assumere che solo utenti autenticati possano inviare input a un servizio: se l'input in realtà risulta accessibile dall'esterno, un attaccante potrebbe sfruttare la debolezza per eseguire attacchi di injection o privilege escalation saltando i controlli.

Domanda 14: Sicurezza vs Ingegneria del Software

Discutere la relazione tra sicurezza e ingegneria del software tradizionale. Quali sono gli aspetti su cui si concentra l'ingegneria del software e come si integra la sicurezza in questo contesto?

Risposta

- **Focus tradizionale** L'ingegneria del software si concentra su funzionalità, manutenibilità, efficienza e affidabilità. Storicamente la sicurezza veniva considerata solo nelle fasi finali di test.
- **Integrazione della sicurezza** Integrare la sicurezza significa adottare pratiche come il threat modeling, code review di sicurezza, fuzzing e vulnerability management durante *tutto* il ciclo di vita del software. Così la sicurezza diventa un requisito non-funzionale fondamentale sin dalla progettazione.

Domanda 15: Principio del Minimo Privilegio

Spiegare il Principio del Minimo Privilegio, perché è importante per la sicurezza del software e come si relaziona con il concetto di isolamento. Fornire un esempio pratico di applicazione di questo principio.

Risposta

- **Concetto e Importanza** Il Principio del Minimo Privilegio afferma che ogni componente o utente deve avere solo i permessi strettamente necessari per svolgere le proprie funzioni. È essenziale per ridurre la superficie d'attacco.
- **Relazione con l'isolamento ed Esempio** Si integra con l'isolamento limitando i danni in caso di compromissione. Ad esempio, l'esecuzione di servizi web in processi separati e con un utente di sistema dai permessi limitatissimi: se il servizio viene compromesso, l'attaccante non può né accedere alle risorse di altri servizi, né ottenere il controllo completo del sistema operativo.

Domanda 16: Fuzzing Roadblock

Descrivere cosa si intende per "fuzzing roadblock" e come possono essere identificati e superati durante una campagna di fuzzing. Fornire esempi concreti di roadblocks comuni e strategie per affrontarli.

Risposta Si parla di "fuzzing roadblock" quando il programma presenta sezioni difficili da superare per il fuzzer, impedendogli di esplorare a fondo il codice.

- **Esempi comuni**
  - Controlli di formato stringenti che richiedono sequenze specifiche di byte
  - Confronti con stringhe o valori magici lunghi (magic bytes)
  - Checksum o hash verification che il fuzzer non sa calcolare
- **Strategie di superamento** In questi casi è necessario superare l'ostacolo fornendo un dizionario ad hoc, usando seed che già contengono le sequenze corrette, oppure disabilitando i controlli complessi (come i checksum) nel target tramite modifiche temporanee al codice durante il fuzzing.

Domanda 17: Deduplicazione dei crash nel fuzzing

Descrivere le tecniche di deduplicazione dei crash utilizzate nel fuzzing, con particolare attenzione allo stack hashing. Quali sono i parametri che influenzano l'efficacia della deduplicazione e quali errori comuni si possono commettere nell'interpretazione dei risultati?

Risposta

- **Stack hashing** Nel fuzzing la deduplicazione raggruppa i crash simili. Lo stack hashing calcola un hash basato sui primi k frame dello stack trace al momento del crash per identificare il bug in modo univoco.
- **Parametri ed errori comuni** Il parametro critico è k: un valore troppo basso causa *sotto-stima* (crash distinti raggruppati), uno troppo alto causa *sovrastima* (crash dello stesso bug visti come separati). Altri fattori includono funzioni di libreria intermedie o le ottimizzazioni del compilatore. Per evitare misinterpretazioni, serve quasi sempre un triage manuale finale.

Domanda 18: Ottimizzazioni del runtime nel fuzzing

Spiegare le principali ottimizzazioni del runtime implementate nei fuzzer moderni come AFL++, descrivendo in dettaglio il funzionamento di fork server, deferred initialization e persistent mode. Quali sono i vantaggi e gli svantaggi di ciascuna tecnica?

Risposta I fuzzer moderni come AFL++ implementano diverse ottimizzazioni runtime per aumentare il throughput.

- **Fork Server:** Inizializza il programma una sola volta e poi usa `fork()` per ogni test case, evitando di ripetere la costosa fase di avvio del SO. Vantaggio: velocità. Svantaggio: non adatto a programmi con stato globale difficile da resettare tra una fork e l'altra.
- **Deferred Initialization:** Permette di forzare il fork *dopo* l'inizializzazione del target (es. caricamento dizionari o config), riducendo l'overhead. Richiede che questa fase iniziale sia totalmente indipendente dall'input.
- **Persistent Mode:** Riutilizza lo *stesso* processo per iterare l'harness su più input tramite un loop infinito in memoria. Vantaggio: estrema velocità. Svantaggio: necessità di ripristinare il target in uno stato pulito al 100% ad ogni loop per evitare memory leak o side effects falsati.

(pagina 71)

Domanda 19: Sicurezza della memoria spaziale vs temporale

Confrontare la sicurezza della memoria spaziale e temporale, descrivendo le principali differenze, i tipi di errori che ciascuna categoria comprende e le tecniche utilizzate per rilevarli e prevenirli.

Risposta

- **Sicurezza Spaziale** Garantisce che ogni accesso avvenga entro i limiti dell'oggetto allocato. Protegge da buffer overflow e underflow su heap, stack o variabili globali (out-of-bounds). Si rileva tramite redzone, shadow memory (ASan), canaries e controlli dei limiti.
- **Sicurezza Temporale** Garantisce che la memoria non venga usata in fasi errate del suo ciclo di vita. Protegge da use-after-free, use-after-return e use-after-scope. Si rileva tramite quarantene della memoria liberata e marcature specifiche in shadow memory.

Domanda 20: Threat-driven design

Spiegare il concetto di "threat-driven design" e come influenza le decisioni di progettazione in ambito di sicurezza. Fornire un esempio concreto di come un modello di minaccia può guidare scelte architetturali specifiche.

Risposta

- **Concetto e Influenza** È un approccio che modella l'architettura basandosi sui rischi concreti: si identificano attaccanti, capacità e obiettivi, sviluppando le difese necessarie. Questo metodo ottimizza le risorse implementando contromisure proporzionate alle minacce più rilevanti.
- **Esempio architetturale** Se il modello di minaccia prevede attacchi automatizzati su un'API pubblica da utenti anonimi, l'architettura privilegerà rate limiting aggressivo, WAF e input validation restrittiva. In una rete isolata interna, le stesse misure avrebbero una priorità architetturale secondaria rispetto ad altre difese interne.

Domanda 21: Quarantena in AddressSanitizer

Descrivere il meccanismo di quarantena utilizzato da AddressSanitizer per rilevare errori di tipo use-after-free. Quali sono i limiti di questo approccio e come potrebbe essere migliorato?

Risposta

- **Meccanismo di Quarantena** Per rilevare use-after-free, i blocchi di memoria appena "liberati" dal programma non vengono restituiti al sistema ma "parcheggiati" in quarantena e marcati come avvelenati nella shadow memory. L'accesso a questi blocchi attiverà subito l'errore in ASan.
- **Limiti e Miglioramenti** La quarantena non ha capacità infinita. Sotto stress di memoria o per allocazioni prolungate, i blocchi più vecchi vengono de-quarantenati e riutilizzati, annullando il rilevamento di use-after-free tardivi. Un miglioramento sarebbe dimensionare dinamicamente la quarantena o usare euristiche per trattenere blocchi "a più alto rischio".

Domanda 22: Fuzzing Harness efficace

Descrivere le caratteristiche di un fuzzing harness efficace e come può influenzare i risultati di una campagna di fuzzing. Fornire un esempio concreto di harness per una libreria di parsing e discutere le considerazioni chiave nella sua implementazione.

Risposta

- **Caratteristiche di un buon harness** Deve essere semplice, isolare bene il codice, evitare costosi side-effect globali e restituire uno stato pulito (cruciale per il persistent mode).
- **Esempio di harness (libxml2)**

```
1 extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
2     if (auto doc = xmlReadMemory((const char *)data, size, "noname.xml", NULL,
       0)) xmlFreeDoc(doc);
3     return 0;
4 }
```

- **Considerazioni chiave** Sopprimere output non critici (es. stderr della libreria) che rallenterebbero l'esecuzione, pulire meticolosamente la memoria (`xmlFreeDoc`) per non esaurire le risorse, e trattare l'input del fuzzer come indipendente e isolato ad ogni esecuzione.

Domanda 23: Seed selection nel fuzzing

Discutere l'importanza della selezione dei seed iniziali nel fuzzing e le strategie per ottimizzare questo processo. Come influisce la qualità dei seed sulla copertura del codice e sull'efficacia complessiva del fuzzing?

Risposta

- **Importanza dei seed** Il fuzzer necessita di punti di partenza (seed). Seed validi e compatti permettono all'algoritmo genetico di saltare complessi controlli preliminari e giungere nel "cuore" del parsing immediatamente.
- **Strategie e Impatto** Le strategie includono: raccolta da test suite reali, minimizzazione corporea (togliere byte superflui al seed preservando la path coverage), e massima diversificazione (usare 5 seed che aprono 5 rami logici, piuttosto che 50 seed quasi uguali). Una buona selezione accelera incredibilmente il tempo di convergenza verso nuovi crash, riducendo le mutazioni a vuoto.

Domanda 24: Isolamento come principio di sicurezza

Spiegare il concetto di isolamento nella sicurezza del software, il ruolo del security monitor e come questo principio viene implementato nei sistemi operativi moderni attraverso la memoria virtuale e le istruzioni privilegiate.

Risposta

- **Concetto e Security Monitor** L'isolamento separa le risorse per contenere i danni di una compromissione. Il security monitor fa rispettare questo confine bloccando le interferenze incrociate tra entità isolate.
- **Implementazione nei SO moderni** Avviene tramite la memoria virtuale: ogni processo gira nel suo "mondo privato", mappato in memoria fisica senza collidere con gli altri. Le operazioni di rete e disco o alterazioni della mappa di memoria richiedono "istruzioni privilegiate" del processore, riservate unicamente al kernel del SO che le valuta per conto dell'utente isolato.

Domanda 25: Analisi della code coverage nel fuzzing

Descrivere i metodi per analizzare e visualizzare la code coverage durante una campagna di fuzzing. Quali strumenti possono essere utilizzati per mappare le informazioni di coverage agli elementi del codice sorgente e come interpretare correttamente questi dati?

Risposta

- **Metodi e Strumenti** La coverage si analizza per capire quanti blocchi/funzioni/edge sono stati toccati dai seed scoperti. Si usano strumenti classici come `gcov/lcov` interfacciati tramite `genhtml` o `afl-cov` per generare report in HTML colorato sul sorgente.
- **Interpretazione dei Dati** L'interprete deve distinguere la coverage superficiale (linee eseguite senza ramificazioni) da quella profonda (logiche complete risolte). Codice molto rosso (non coperto) indica dove la strategia di mutazione fallisce (roadblock), aiutando il ricercatore a costruire un dizionario mirato.

Domanda 26: Confronto tra sanitizer

Confrontare UndefinedBehaviorSanitizer (UBSan), AddressSanitizer (ASan) e MemorySanitizer (MSan) in termini di overhead di esecuzione, tipi di errori rilevati e compatibilità con altri strumenti. Quando è preferibile utilizzare uno rispetto agli altri?

Risposta

- **UBSan:** Rileva undefined behaviors matematici e logici (overflow interi, shift esagerati, div by zero). Ha un overhead bassissimo e convive bene con chiunque (solitamente con ASan).

- **ASan:** Rileva mem safety spaziale e (in parte) temporale (OOB, UAF). Ha un overhead moderato (2x) per via delle redzones. Si usa ovunque.
- **MSan:** Rileva esclusivamente UUM (letture/utilizzi di memoria non inizializzata). Ha un overhead molto elevato (3x), non è compatibile con ASan e richiede librerie esterne totalmente ricompilate in MSan.

La scelta dipende: UBSan per logica a costo 0, ASan per il day-to-day bug hunting spaziale, MSan specificamente per debug di leak e valori "spazzatura".

Domanda 27: Modelli di minaccia in contesti diversi

Descrivere come i modelli di minaccia possono variare in contesti applicativi diversi (es. applicazioni web, sistemi embedded, applicazioni desktop). Fornire esempi concreti di minacce specifiche per ciascun contesto e come queste influenzano le strategie di mitigazione.

Risposta

- **Applicazioni Web:** Minacce su attacchi di input (SQLi, XSS) o furti di identità. Mitigazioni: Sanitizzazione rigida, cookie secure, WAF.
- **Sistemi Embedded:** L'attaccante può avere accesso fisico al dispositivo. Minacce su JTAG, estrazione firmware, attacchi side-channel. Mitigazioni: Secure boot, hardware anti-tamper, cifratura firmware.
- **Applicazioni Desktop:** Malware utente, file ostili da parser, privilege escalation verso il SO. Mitigazioni: Sandboxing stretto dell'app (es. app store constraints), drop dei privilegi di SO, auto-update continui.

Domanda 28: Favored inputs nel fuzzing

Spiegare il concetto di "favored inputs" in AFL/AFL++ e come contribuisce all'efficienza del processo di fuzzing. Descrivere l'algoritmo utilizzato per selezionare gli input favoriti e il suo impatto sulla copertura del codice.

Risposta

- **Concetto e Algoritmo** I "favored inputs" sono specifici seed in coda scelti per essere sproporzionatamente favoriti dal motore mutazionale. L'algoritmo li designa valutando quali seed scoprono un edge unico (che nessun altro copre) in relazione al loro peso (i più veloci e piccoli tra quelli utili).
- **Impatto e Efficienza** Anziché fuzzare il milionesimo seed inutile e massiccio, AFL concentra CPU/Cicli sui favored. Questo abbassa drasticamente il "rumore" esplorativo, convergiendo in tempi molto più brevi alla scoperta di codice interno nascosto.